

xv6-2020

Lab4 Traps

1 RISC-V assembly

1.a0-a7 用于保存函数的参数，13保存在 a2 中。

2.没有调用，函数 f 和 g 直接被内联优化了。

```
void main(void) {  
    1c: 1141          addi    sp,sp,-16  
    1e: e406          sd     ra,8(sp)  
    20: e022          sd     s0,0(sp)  
    22: 0800          addi    s0,sp,16  
    printf("%d %d\n", f(8)+1, 13);  
    24: 4635          li     a2,13  
    26: 45b1          li     a1,12  
    28: 00000517      auipc   a0,0x0  
}
```

3.0x640。

0000000000000630 <printf>:

```
void  
printf(const char *fmt, ...)  
{  
    630: 711d          addi    sp,sp,-96  
}
```

4.ra保存的是函数调用后返回的地址，为0x38。

```
void main(void) {  
    1c: 1141          addi    sp,sp,-16  
    1e: e406          sd     ra,8(sp)  
    20: e022          sd     s0,0(sp)  
    22: 0800          addi    s0,sp,16  
    printf("%d %d\n", f(8)+1, 13);  
    24: 4635          li     a2,13  
    26: 45b1          li     a1,12  
    28: 00000517      auipc   a0,0x0  
    2c: 7b050513      addi    a0,a0,1968 # 7d8 <malloc+0xea>  
    30: 00000097      auipc   ra,0x0  
    34: 600080e7      jalr    1536(ra) # 630 <printf>  
    exit(0);  
    38: 4501          li     a0,0  
}
```

5.He110 World.

6.y 的输出结果为 0，保存的是 a2 寄存器中的值。

2 Backtrace

该题目的目的为实现 backtrace 函数，以打印当前进程的调用链。

首先，在 kernel/riscv.h 文件中添加读取栈帧指针的函数，执行 `mv %0, s0` 将 s0 寄存器的值读出，s0 寄存器用于存放帧指针：

```
static inline uint64
r_fp()
{
    uint64 x;
    asm volatile("mv %0, s0" : "=r" (x) );
    return x;
}
```

然后，在 kernel/printf.c 中添加 backtrace 函数。该函数首先调用 r_fp 获取栈帧指针的首地址，向上对齐获得栈顶帧指针之后，在循环中打印，return address 是当前 fp - 8，调用该函数的栈帧是 fp - 16：

```
void
backtrace(void)
{
    uint64 fpaddr = r_fp();
    uint64 max = PGROUNDUP(fpaddr);
    while (fpaddr < max) {
        printf("%p\n", *((uint64*)(fpaddr - 8)));
        fpaddr = *((uint64*)(fpaddr - 16));
    }
}
```

之后，在 kernel/defs.h 添加 backtrace 的声明，在 kernel/syspro.c 的 sys_sleep 函数和 kernel/printf.c 的 panic 函数中添加对 backtrace 的调用即可。

3 Alarm

该题的目的是让我们增加系统调用，该系统调用会在指定时间间隔后调用用户态的函数。首先，在 kernel/proc.h 的 proc 结构体中添加，ticks 用来保存定时器的周期，alarmhandler 用来存储定时器处理函数的地址，tickspass 用于表明定时器状态，accessible 用于表明是否在进行信号处理，alarmtrapframe 用于保存原有的上下文：

```
int ticks;
uint64 alarmhandler;
int tickspass;
struct trapframe *alarmtrapframe;
int accessible;
```

接着，在 kernel/proc.c 中添加分配和回收 alarmtrapframe 的代码。分配和初始化：

```
if((p->alarmtrapframe = (struct trapframe *)kalloc()) == 0){
    release(&p->lock);
    return 0;
}
```

```
p->ticks = 0;
p->alarmhandler = 0;
p->tickspass = 0;
p->accessible = 1;
```

回收：

```

if(p->trapframe)
    kfree((void*)p->trapframe);
p->trapframe = 0;
if(p->alarmtrapframe)
    kfree((void*)p->alarmtrapframe);
p->alarmtrapframe=0;

```

然后，在 kernel/syscall.h 中添加调用号：

```

#define SYS_sigalarm 22
#define SYS_sigreturn 23

```

在 kernel/syscall.c 中添加系统调用映射：

```

extern uint64 sys_sigalarm(void);
extern uint64 sys_sigreturn(void);

[SYS_close]    sys_close,
[SYS_sigalarm] sys_sigalarm,
[SYS_sigreturn] sys_sigreturn,

```

然后，在 kernel/sysproc.c 中添加系统调用，其中，sys_sigalarm 为当前进程设置一个定时器，该定时器会在每次经过指定的 ticks 数量后触发，并调用用户程序提供的 alarmhandler 函数作为处理程序：

```

uint64
sys_sigalarm(void)
{
    int ticks = 0;
    uint64 alarmhandler = 0;

    if(argint(0, &ticks) < 0 || argaddr(1, &alarmhandler) < 0)
        return -1;

    struct proc *p = myproc();
    p->ticks = ticks;
    p->alarmhandler = alarmhandler;
    p->tickspass = 0;
    p->accessable = 1;

    return 0;
}

```

sys_return 用于恢复信号处理后的进程状态，并恢复到进程在处理信号之前的执行状态。首先判断信号处理是否已经完成并且可以恢复状态，如果可以恢复，memmove 函数将 p->alarmtrapframe 中保存的进程状态恢复到 p->trapframe 中，并将信号处理标志位置为可以处理：

```

uint64
sys_sigreturn(void)
{
    struct proc *p = myproc();
    if (p->accessable == 0) {
        memmove(p->trapframe, p->alarmtrapframe, sizeof(struct trapframe));
        p->accessable = 1;
    }

    return 0;
}

```

然后，在 kernel/trap.c 的 usertrap 函数中，每当定时器中断触发时，进程的 tickspass 计数器增加 1，如果 p->ticks 不为 0 且 tickspass 达到指定的周期（即 tickspass == p->ticks），则表示定时器触发了，应该执行定时器处理逻辑，此时，将

tickspass 计数器重置为 0，准备下一个周期的计数，接着检查进程是否可以访问其上下文 (`p->accessible==1`)，如果可以，则保存当前进程的上下文到 `p->alarmtrapframe` 中，再将进程的 `trapframe` 中的 `pc (epc)` 设置为 `alarmhandler` 的地址，即定时器触发时将跳转到用户程序提供的定时器处理函数，最后将 `accessible` 设置为 0，表示信号处理程序不可重复访问，防止在信号处理期间再次修改上下文：

```
// give up the CPU if this is a timer interrupt.
if(which_dev == 2) {
    p->tickspass++;
    if (p->ticks != 0 && p->tickspass == p->ticks) {
        p->tickspass = 0;
        if (p->accessible == 1) {
            memmove(p->alarmtrapframe, p->trapframe, sizeof(struct trapframe));
            p->trapframe->epc = p->alarmhandler;
            p->accessible = 0;
        }
    } else {
        yield();
    }
}
```

最后，在 `user/user.h` 和 `user/usys.pl` 中添加系统调用声明和入口即可：

```
int sigalarm(int ticks, void (*handler)());
int sigreturn(void);

entry("sigalarm");
entry("sigreturn");
```

4 Test

`make grade` 可以通过全部测试：

```
== Test answers-traps.txt == answers-traps.txt: OK
== Test backtrace test ==
$ make qemu-gdb
backtrace test: OK (1.9s)
== Test running alarmtest ==
$ make qemu-gdb
(3.2s)
== Test alarmtest: test0 ==
alarmtest: test0: OK
== Test alarmtest: test1 ==
alarmtest: test1: OK
== Test alarmtest: test2 ==
alarmtest: test2: OK
== Test usertests ==
$ make qemu-gdb
usertests: OK (47.1s)
== Test time ==
time: OK
Score: 85/85
```

Lab5 Lazy allocation

该次实验是为了在 `xv6` 中实现惰性分配，即给进程分配内存的时候，只先改变指针地址，并不分配物理内存，只有当使用这块内存时才引发缺页异常重新分配内存。

1 Eliminate allocation from `sbrk()`

kernel/sysproc.c 中，从 sbrk() 系统调用中将页面分配的功能删除，即删除进程空间增长的调用，只是将进程的大小增加 n 并返回旧大小，这里同时处理了 $n < 0$ 的情况，参照

growproc() :

```
uint64
sys_sbrk(void)
{
    int addr;
    int n;

    if(argint(0, &n) < 0)
        return -1;
    // if(growproc(n) < 0)
    //     return -1;
    addr = myproc()->sz;
    myproc()->sz += n;
    if (n < 0) {
        uvmdealloc(myproc()->pagetable, addr, addr + n);
    }
    return addr;
}
```

2 Lazy allocation

kernel/trap.c 中，在 usertrap() 函数中添加对缺页异常的处理。首先通过 r_stval() 返回发生缺页异常的虚拟地址，接着检查该虚拟地址是否在进程的有效地址空间范围内，若超出了，进程将被标记为 killed，否则分配一页物理内存（调用 kalloc() 分配），并返回指向该物理内存的指针 pa，初始化分配的页，并将该物理页面 pa 映射到虚拟地址 va，完成页面的分配。

```
},
} else if((which_dev = devintr()) != 0){
    // ok
} else if (r_scause() == 13 || r_scause() == 15) {
    uint64 va = r_stval();
    if(va >= p->sz || va <= PGROUNDDOWN(p->trapframe->sp)) {
        p->killed = 1;
    } else {
        void *pa = kalloc();
        if(pa == 0){
            p->killed = 1;
        } else {
            memset(pa, 0, PGSIZE);
            if(mappages(p->pagetable, PGROUNDDOWN(va), PGSIZE, (uint64)pa, PTE_W | PTE_X | PTE_R | PTE_U) != 0) {
                kfree(pa);
                p->killed = 1;
            }
        }
    }
}
} else {
    printf("usertrap(): unexpected scause %p pid=%d\n", r_scause(), p->pid);
    printf("          sepc=%p stval=%p\n", r_sepc(), r_stval());
    p->killed = 1;
}
}
```

3 Lazytests and Usertests

如果 sbrk 增加 p->sz 后直到释放进程都没有真正使用某些地址，那么这些地址就不会分配到内存，以及在页表中添加映射。但 uvmunmap() 会根据 p->sz 来在页表中取消映射；fork 时会使用的 uvmcopy() 会根据 p->sz 来照搬父进程的页表给子进程。故修改

kernel/vm.c 中的 `uvmunmap()` , `uvmcopy()` 函数, 遇到未映射的页时直接直接 `continue` :

```
for(a = va; a < va + npages*PGSIZE; a += PGSIZE){
    if((pte = walk(pagetable, a, 0)) == 0)
        // panic("uvmunmap: walk");
        continue;
    if((*pte & PTE_V) == 0)
        // panic("uvmunmap: not mapped");
        continue;
```

处理进程将有效地址从 `sbrk()` 传递给系统调用 (read or write), 但尚未分配该地址的内存的情况, 和 `usertrap()` 中处理缺页异常一致, 将类似过程写入 `kernel/vm.c` 中的 `walkaddr()` 函数中即可:

```
pte = walk(pagetable, va, 0);
if(pte == 0 || (*pte & PTE_V) == 0) {
    struct proc *p = myproc();
    if(va >= p->sz || va <= PGROUNDDOWN(p->trapframe->sp)) {
        return 0;
    } else {
        void *pa = kalloc();
        if(pa == 0) {
            return 0;
        } else {
            memset(pa, 0, PGSIZE);
            if(mappages(p->pagetable, PGROUNDDOWN(va), PGSIZE, (uint64)pa, PTE_W | PTE_X | PTE_R | PTE_U)!=0) {
                kfree(pa);
                return 0;
            }
            return (uint64)pa;
        }
    }
}
if((*pte & PTE_U) == 0)
    return 0;
```

4 Test

make grade 可以通过全部测试：

```
== Test running lazytests ==
$ make qemu-gdb
(3.5s)
== Test lazy: map ==
lazy: map: OK
== Test lazy: unmap ==
lazy: unmap: OK
== Test usertests ==
$ make qemu-gdb
(73.0s)
== Test usertests: pgbug ==
usertests: pgbug: OK
== Test usertests: sbrkbugs ==
usertests: sbrkbugs: OK
== Test usertests: argptest ==
usertests: argptest: OK
== Test usertests: sbrkmuch ==
usertests: sbrkmuch: OK
== Test usertests: sbrkfail ==
usertests: sbrkfail: OK
== Test usertests: sbrkarg ==
usertests: sbrkarg: OK
== Test usertests: stacktest ==
usertests: stacktest: OK
== Test usertests: execout ==
usertests: execout: OK
== Test usertests: copyin ==
usertests: copyin: OK
== Test usertests: copyout ==
usertests: copyout: OK
== Test usertests: copyinstr1 ==
usertests: copyinstr1: OK
== Test usertests: copyinstr2 ==
usertests: copyinstr2: OK
== Test usertests: copyinstr3 ==
usertests: copyinstr3: OK
== Test usertests: rwsbrk ==
usertests: rwsbrk: OK
== Test usertests: truncate1 ==
usertests: truncate1: OK
== Test usertests: truncate2 ==
usertests: truncate2: OK
== Test usertests: truncate3 ==
usertests: truncate3: OK
== Test usertests: reparent2 ==
usertests: reparent2: OK
== Test usertests: badarg ==
usertests: badarg: OK
== Test usertests: reparent ==
usertests: reparent: OK
== Test usertests: twochildren ==
usertests: twochildren: OK
```

```

== Test   usertests: forkfork ==
    usertests: forkfork: OK
== Test   usertests: forkforkfork ==
    usertests: forkforkfork: OK
== Test   usertests: createdelete ==
    usertests: createdelete: OK
== Test   usertests: linkunlink ==
    usertests: linkunlink: OK
== Test   usertests: linktest ==
    usertests: linktest: OK
== Test   usertests: unlinkread ==
    usertests: unlinkread: OK
== Test   usertests: concreate ==
    usertests: concreate: OK
== Test   usertests: subdir ==
    usertests: subdir: OK
== Test   usertests: fourfiles ==
    usertests: fourfiles: OK
== Test   usertests: sharedfd ==
    usertests: sharedfd: OK
== Test   usertests: exectest ==
    usertests: exectest: OK
== Test   usertests: bigargtest ==
    usertests: bigargtest: OK
== Test   usertests: bigwrite ==
    usertests: bigwrite: OK
== Test   usertests: bsstest ==
    usertests: bsstest: OK
== Test   usertests: sbrkbasic ==
    usertests: sbrkbasic: OK
== Test   usertests: kernmem ==
    usertests: kernmem: OK
== Test   usertests: validatetest ==
    usertests: validatetest: OK
== Test   usertests: opentest ==
    usertests: opentest: OK
== Test   usertests: writetest ==
    usertests: writetest: OK
== Test   usertests: writebig ==
    usertests: writebig: OK
== Test   usertests: createtest ==
    usertests: createtest: OK
== Test   usertests: openiput ==
    usertests: openiput: OK
== Test   usertests: exitiput ==
    usertests: exitiput: OK
== Test   usertests: iput ==
    usertests: iput: OK

== Test   usertests: mem ==
    usertests: mem: OK
== Test   usertests: pipe1 ==
    usertests: pipe1: OK
== Test   usertests: preempt ==
    usertests: preempt: OK
== Test   usertests: exitwait ==
    usertests: exitwait: OK
== Test   usertests: rmdot ==
    usertests: rmdot: OK
== Test   usertests: fourteen ==
    usertests: fourteen: OK
== Test   usertests: bigfile ==
    usertests: bigfile: OK
== Test   usertests: dirfile ==
    usertests: dirfile: OK
== Test   usertests: iref ==
    usertests: iref: OK
== Test   usertests: forktest ==
    usertests: forktest: OK
== Test time ==
time: OK
Score: 119/119

```

Lab6 Copy on-write

该次实验目的是设计 copy-on-write fork()，以延迟为子进程分配和复制物理内存页面，直到实际需要。

1 Implement copy-on write

首先，在 `kernel/riscv.h` 中给页表添加新的标志位，代表这个页是否是写时复制页，称为 `PTE_COW`，使用保留位：

```
#define PTE_COW (1L << 8)
```

接着，写时复制是写的时候才复制父进程的物理页给子进程，所以在 `kernel/vm.c` 的 `uvmcopy()` 函数中，去除复制内存的代码，清除父进程中所有 PTE 的 `PTE_W` 位，并且设置 `PTE_COW` 位（如果该页本身不是只读的），并将父进程映射的物理页直接映射到子进程中，权限保持和父进程一致，调用 `krefpage()` 增加页的引用计数：

```
for(i = 0; i < sz; i += PGSIZE){
    if((pte = walk(old, i, 0)) == 0)
        panic("uvmcopy: pte should exist");
    if((*pte & PTE_V) == 0)
        panic("uvmcopy: page not present");
    pa = PTE2PA(*pte);

    if (*pte & PTE_W)
        *pte = (*pte & ~PTE_W) | PTE_COW;

    flags = PTE_FLAGS(*pte);
    if (mappages(new, i, PGSIZE, (uint64)pa, flags) != 0)
        goto err;

    krefpage((void*)pa);
}
return 0;
```

引用计数是为了避免内存泄漏，当多个进程映射同一个物理页的时候，只记录该物理页的引用数，调用 `kfree` 释放物理页的时候，只对引用数减一，当引用数为 0 时，释放物理页。然后，在 `kernel/trap.c` 中的函数 `usertrap()` 中添加对页面错误的检测，如果当前访问的地址符合 `COW` 页的条件，进行进程复制内存操作：

```
} else if ((r_scause() == 13 || r_scause() == 15) && uvmcheckcowpage(r_stval())) {
    if (uvmcowcopy(r_stval()) == -1)
        p->killed = 1;
```

其中，通过调用 `r_scause` 判断是否是页面错误，实现 `uvmcheckcowpage()` 函数判断是否是 `COW` 导致的页面错误，符合这些条件，调用实现的 `uvmcowcopy()` 函数执行写时复制操作。

`kernel/vm.c` 中的 `uvmcheckcowpage()` 函数如下，判断地址是否进程内存范围内、地址是否有映射、地址是否有效且是 `COW` 页：

```
int uvmcheckcowpage(uint64 va) {
    pte_t* pte;
    struct proc* p = myproc();

    return va < p->sz
        && ((pte = walk(p->pagetable, va, 0)) != 0)
        && (*pte & PTE_V)
        && (*pte & PTE_COW);
}
```

`kernel/vm.c` 中的 `uvmcowcopy()` 函数如下，先获取虚拟地址的页表项、映射的物理地址、新分配的物理页，再修改新的映射，若当前页有 `COW` 标识，权限修改为可写，清除 `COW` 标识，最后清除旧的映射，建立新的映射：

```

int uvmcowcopy(uint64 va) {
    pte_t* pte;
    struct proc* p = myproc();

    if ((pte = walk(p->pagetable, va, 0)) == 0)
        panic("uvmcowcopy: walk");

    uint64 pa = PTE2PA(*pte);
    uint64 new = (uint64)kcopy_n_deref((void*)pa);
    if (new == 0)
        return -1;

    uint64 flags = PTE_FLAGS(*pte);
    if (*pte & PTE_COW)
        flags = (PTE_FLAGS(*pte) | PTE_W) & ~PTE_COW;
    uvmunmap(p->pagetable, PGROUNDDOWN(va), 1, 0);
    if (mappages(p->pagetable, va, 1, new, flags) == -1)
        panic("uvmcowcopy: mappages");

    return 0;
}

```

其中，`kcopy_n_deref()` 函数功能是，当有进程通过 `cow` 复制新的物理页时，将原本物理页的引用数-1，返回新创建的物理页地址。若原本的物理页引用数为1，直接使用该物理页。

`kernel/vm.c` 中的 `copyout()` 函数中，检测复制的页是否是 `cow` 页，若是，执行复制操作：

```

while(len > 0){
    if (uvmcheckcowpage(dstva))
        uvmcowcopy(dstva);
    va0 = PGROUNDDOWN(dstva);
    pa0 = walkaddr(pagetable, va0);
}

```

最后，处理内存分配回收时的操作。在 `kernel/kalloc.c` 中定义宏，分别用于由物理地址获取物理页id、获取物理页数上限、获取地址对应物理页引用数：

```

#define PA2PGREF_ID(p) (((p)-KERNBASE)/PGSIZE)
#define PGREF_MAX_ENTRIES PA2PGREF_ID(PHYSTOP)
#define PA2PGREF(p) pageref[PA2PGREF_ID((uint64)(p))]

```

再定义每个物理页的引用数数组和用于该数组的锁，防止竞态引起内存泄漏：

```

int pageref[PGREF_MAX_ENTRIES];
struct spinlock pgreflock;

```

在 `kinit()` 函数中进行锁的初始化：

```

void
kinit()
{
    initlock(&kmem.lock, "kmem");
    initlock(&pgreflock, "pgref");
    freerange(end, (void*)PHYSTOP);
}

```

在 `kfree()` 函数中，仅当页面的引用数等于 0 时释放页面：

```

void
kfree(void *pa)
{
    struct run *r;

    if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
        panic("kfree");

    acquire(&pgreflock);
    if (--PA2PGREF(pa) <= 0) {
        // Fill with junk to catch dangling refs.
        memset(pa, 1, PGSIZE);

        r = (struct run*)pa;

        acquire(&kmem.lock);
        r->next = kmem.freelist;
        kmem.freelist = r;
        release(&kmem.lock);
    }
    release(&pgreflock);
}

```

修改 `kalloc()` 函数，将新分配的物理页的引用数设置为1:

```

void *
kalloc(void)
{
    struct run *r;

    acquire(&kmem.lock);
    r = kmem.freelist;
    if(r)
        kmem.freelist = r->next;
    release(&kmem.lock);

    if (r) {
        memset((char*)r, 5, PGSIZE); // fill with junk
        PA2PGREF(r) = 1;
    }
    return (void*)r;
}

```

实现 `krefpage()`，用于增加页的引用数:

```

void krefpage(void* pa) {
    acquire(&pgreflock);
    PA2PGREF(pa)++;
    release(&pgreflock);
}

```

实现 `kcopy_n_deref()`，用于返回对应的物理页，当前物理页的引用数为1，无需分配新的物

理页，否则分配新的物理页，并把旧页中的数据复制到新页，再将旧页引用数减一：

```
void* kcopy_n_deref(void* pa) {
    acquire(&pgreflock);

    if (PA2PGREF(pa) <= 1) {
        release(&pgreflock);
        return pa;
    }

    uint64 newpa = (uint64)kalloc();
    if (newpa == 0) {
        release(&pgreflock);
        return 0;
    }
    memmove((void*)newpa, (void*)pa, PGSIZE);

    PA2PGREF(pa)--;

    release(&pgreflock);
    return (void*)newpa;
}
```

2 Test

make grade 可以通过全部测试：

```
== Test running cowtest ==
$ make qemu-gdb
(5.4s)
== Test    simple ==
simple: OK
== Test    three ==
three: OK
== Test    file ==
file: OK
== Test usertests ==
$ make qemu-gdb
(78.2s)
== Test    usertests: copyin ==
usertests: copyin: OK
== Test    usertests: copyout ==
usertests: copyout: OK
== Test    usertests: all tests ==
usertests: all tests: OK
== Test time ==
time: OK
Score: 110/110
```

Lab7 Multithreading

该实验要求实现用户级线程、优化并发程序，使用线程提速程序，实现同步屏障。

1 Uthread: switching between threads

该任务要求实现创建线程和保存/恢复寄存器以在线程之间切换的功能。

首先，仿照 swtch.S 实现 uthread_switch.S，完成上下文切换，保存当前线程寄存器，读取新线程的寄存器：

```

thread_switch:
    /* YOUR CODE HERE */

    sd ra, 0(a0)
    sd sp, 8(a0)
    sd s0, 16(a0)
    sd s1, 24(a0)
    sd s2, 32(a0)
    sd s3, 40(a0)
    sd s4, 48(a0)
    sd s5, 56(a0)
    sd s6, 64(a0)
    sd s7, 72(a0)
    sd s8, 80(a0)
    sd s9, 88(a0)
    sd s10, 96(a0)
    sd s11, 104(a0)

    ld ra, 0(a1)
    ld sp, 8(a1)
    ld s0, 16(a1)
    ld s1, 24(a1)
    ld s2, 32(a1)
    ld s3, 40(a1)
    ld s4, 48(a1)
    ld s5, 56(a1)
    ld s6, 64(a1)
    ld s7, 72(a1)
    ld s8, 80(a1)
    ld s9, 88(a1)
    ld s10, 96(a1)
    ld s11, 104(a1)

    ret    /* return to ra */

```

在 user/uthread.c 中，给线程增加 context 结构体，用于保存线程上下文，并在线程结构体中加入 context 结构体：

```

struct context {
    uint64 ra;
    uint64 sp;

    uint64 s0;
    uint64 s1;
    uint64 s2;
    uint64 s3;
    uint64 s4;
    uint64 s5;
    uint64 s6;
    uint64 s7;
    uint64 s8;
    uint64 s9;
    uint64 s10;
    uint64 s11;
};

struct thread {
    char    stack[STACK_SIZE]; /* the thread's stack */
    int     state;              /* FREE, RUNNING, RUNNABLE */
    struct context context;
};

```

在调度函数 thread_schedule 中加入线程切换函数 thread_switch：

```

if (current_thread != next_thread) { /* switch threads? */
    next_thread->state = RUNNING;
    t = current_thread;
    current_thread = next_thread;
    /* YOUR CODE HERE
    * Invoke thread_switch to switch from t to next_thread:
    * thread_switch(??, ??);
    */
    thread_switch((uint64)&t->context, (uint64)&next_thread->context);
}

```

在线程创建函数 `thread_create()` 中，分别设置返回地址 `pa` 和栈指针 `sp`：

```
void
thread_create(void (*func)())
{
    struct thread *t;

    for (t = all_thread; t < all_thread + MAX_THREAD; t++) {
        if (t->state == FREE) break;
    }
    t->state = RUNNABLE;
    // YOUR CODE HERE
    t->context.ra = (uint64)func;
    t->context.sp = (uint64)&t->stack + (STACK_SIZE - 1);
}
```

2 Using threads

该任务要求实现线程安全访问，需在并发写共享数据的地方加锁。

首先，在 `notxv6/ph.c` 声明线程锁，给每一个散列桶声明一个锁：

```
pthread_mutex_t lock[NBUCKET];
```

在 `main()` 中初始化锁：

```
for (int i = 0; i < NBUCKET; ++i) {
    pthread_mutex_init(&lock[i], NULL);
}
```

在执行 `put` 操作的时候，针对散列桶上锁：

```
static
void put(int key, int value)
{
    int i = key % NBUCKET;
    pthread_mutex_lock(&lock[i]);

    // is the key already present?
    struct entry *e = 0;
    for (e = table[i]; e != 0; e = e->next) {
        if (e->key == key)
            break;
    }
    if (e) {
        // update the existing key.
        e->value = value;
    } else {
        // the new is new.
        insert(key, value, &table[i], table[i]);
    }

    pthread_mutex_unlock(&lock[i]);
}
```

3 Barrier

该任务要求实现屏障。

线程调用 `barrier` 之后，`bstate` 中的线程数 `nthread` 应该增一，判断当前进入屏障的线程数是否达到全局的 `nthread`。若未达到，调用 `pthread_cond_wait`，让当前线程睡眠，等待线程数达到要求；若达到了，将 `bstate` 中的轮数 `round` 增一、清空 `bstate` 的线程数并唤醒其

他正在睡眠的线程，据此可以完成 `barrier()`：

```
static void
barrier()
{
    // YOUR CODE HERE
    //
    // Block until all threads have called barrier() and
    // then increment bstate.round.
    //
    pthread_mutex_lock(&bstate.barrier_mutex);
    if (++bstate.nthread < nthread)
        pthread_cond_wait(&bstate.barrier_cond, &bstate.barrier_mutex);
    else {
        bstate.nthread = 0;
        bstate.round++;
        pthread_cond_broadcast(&bstate.barrier_cond);
    }
    pthread_mutex_unlock(&bstate.barrier_mutex);
}
```

4 Test

`make grade` 可以通过全部测试：

```
== Test uthread ==
$ make qemu-gdb
uthread: OK (2.1s)
== Test answers-thread.txt == answers-thread.txt: OK
== Test ph_safe == make[1]: Entering directory '/home/jzd/xv6-labs-2020'
make[1]: 'ph' is up to date.
make[1]: Leaving directory '/home/jzd/xv6-labs-2020'
ph_safe: OK (10.4s)
== Test ph_fast == make[1]: Entering directory '/home/jzd/xv6-labs-2020'
make[1]: 'ph' is up to date.
make[1]: Leaving directory '/home/jzd/xv6-labs-2020'
ph_fast: OK (26.0s)
== Test barrier == make[1]: Entering directory '/home/jzd/xv6-labs-2020'
make[1]: 'barrier' is up to date.
make[1]: Leaving directory '/home/jzd/xv6-labs-2020'
barrier: OK (3.1s)
== Test time ==
time: OK
Score: 60/60
```